

RHEINISCHE HOCHSCHULE KÖLN

University of Applied Sciences

Studiengang: Informatik (Bachelor of Science)



Transfer-Projekt

Entwicklung einer konfigurierbaren Streaming-Landingpage mit Community-Integration

Vorgelegt von: Robin Wagner

653242019

Dozent: Prof. Dr. Johannes Mauer

Sommersemester 2026

Inhaltsverzeichnis

1 Themenstellung	3
1.1 Allgemeine Vorstellung und Eingrenzung der Thematik	3
1.2 Konkrete Aufgabenstellung	3
1.3 Vorgehensweise und Methodik	3
1.4 Zielsetzung und Anforderungen	4
2 Das Baukasten-Prinzip	5
2.1 Konfigurationsebene 1 – siteConfig.ts	5
2.2 Konfigurationsebene 2 – i18n-Sprachdateien.....	5
2.3 Setup-Assistent	6
3 Tech Stack	6
3.1 React & TypeScript	6
3.2 Vite	7
3.3 Supabase	7
3.4 Bun (TwitchAddon)	7
3.5 Docker & Nginx	8
4 Entwicklung	8
4.1 Systemarchitektur	8
4.1.1 Frontend-Aufbau	10
4.1.2 Backend & Datenmodell	10
4.1.3 TwitchAddon (Bot und Panel-Extension)	10
4.1.4 Automatisierung über GitHub Actions und Edge Functions	11
4.2 Design	12

4.3 Hosting und CI/CD-Pipeline.....	13
5 Tests.....	14
5.1 Unit-Tests	14
5.2 User-Tests.....	14
5.3 Performance-Prüfung mit PageSpeed Insights	14
5.4 Sicherheitsprüfung	15
5.4.1 npm audit.....	15
5.4.2 Retire.js	15
5.4.3 Renovate	16
6 Setup.....	16
7 Projektabschluss.....	17
7.1 Fazit.....	17
7.2 Herausforderungen	17
7.3 Zukunftspotential.....	18
8 Eigenständigkeitserklärung	19

1 Themenstellung

1.1 Allgemeine Vorstellung und Eingrenzung der Thematik

Das übergeordnete Thema ist die Entwicklung einer individuell konfigurierbaren Landingpage für Twitch-Streamer, die als zentrale Anlaufstelle für die Community dient. Solche Seiten sind im Streaming-Umfeld weit verbreitet, bestehen jedoch häufig aus zusammenhanglosen Einzellösungen ohne einheitliche Datenbasis oder Anpassungsmöglichkeiten.

Der aktuelle Entwicklungsstand zeigt ein wachsendes Angebot an Plattformen wie StreamElements oder Linktree, die jedoch entweder auf Monetarisierung ausgelegt oder technisch inflexibel sind. Das Projekt setzt deshalb auf eine selbst gehostete, quelloffene Lösung auf Basis moderner Webtechnologien (React, TypeScript, Vite, Supabase), ergänzt durch einen Discord-Bot und ein Twitch-Addon für eine nahtlose Community-Anbindung. Im Fokus steht von Beginn an ein Baukasten-Gedanke: Die Plattform soll nicht nur einen einzelnen Kanal bedienen, sondern als wiederverwendbare Vorlage für beliebige weitere Streamer dienen.

1.2 Konkrete Aufgabenstellung

In dieser Arbeit soll ein wiederverwendbarer Baukasten für Streaming-Landingpages konzipiert, implementiert und produktiv erprobt werden. Der Twitch-Kanal `hd1920x1080` dient dabei als Pilotkanal: An ihm wird der Baukasten unter realen Bedingungen mit echter Community erprobt, ohne dass die Lösung selbst kanalspezifisch wird. Funktional muss der Baukasten typische Streamer-Bedürfnisse abdecken – integrierter Chat, Spenden-System mit Betragsvorgaben, eine interaktive Overlay-Steuerung (Twitch-Addon) sowie weitere Community-Funktionen.

Das zentrale Problem ist die Vereinheitlichung verschiedener Dienste (Twitch, Discord, Supabase) in einer konsistenten, wartbaren Applikation, ohne auf proprietäre Plattformen angewiesen zu sein. Konkret soll beantwortet werden, wie eine solche Plattform als modularer Baukasten realisiert werden kann, sodass Inhalte und Komponenten einfach angepasst oder erweitert werden können – idealerweise so, dass ein neuer Streamer ohne Eingriff in den Quellcode auskommt.

1.3 Vorgehensweise und Methodik

Die Vorgehensweise folgt einem iterativen Entwicklungsansatz. Zunächst werden Anforderungen auf Basis des realen Einsatzkontexts (Twitch-Kanal, Discord-Community)

erhoben und in GitHub Issues überführt. Anschließend wird die technische Architektur geplant: Frontend mit React/TypeScript/Vite, Backend-Datenhaltung über Supabase (PostgreSQL) sowie ein Twitch-Addon für den Einsatz im Stream-Overlay. Hinzu kommt ein Discord-Benachrichtigungsdienst, um Informationen der Seite zu verteilen, da sich Discord in der Branche als Standard etabliert hat.

Ein zentrales Ziel der Architekturentscheidungen ist es, die Betriebskosten auf ein Minimum zu reduzieren. Das Hosting wird so gestaltet, dass abgesehen von der Domain keine laufenden Kosten entstehen. Das Frontend wird auf GitHub Pages deployed, die Datenbankschicht über den gebührenfreien Supabase-Plan betrieben und der Discord-Benachrichtigungsdienst per Supabase Edge Function gehostet. Durch die Kombination dieser Dienste entsteht eine vollständig betriebsfähige Infrastruktur ohne monatliche Ausgaben.

Datensicherheit ist ein weiterer wichtiger Punkt. Sensible Daten wie API-Keys und Datenbankzugänge werden ausschließlich über Umgebungsvariablen verwaltet und sind nicht im Repository enthalten. Eine `.env.example`-Datei dokumentiert die benötigten Variablen ohne reale Werte. Auf Serverseite werden Secrets über die Umgebungsvariablen-Verwaltung von Github gespeichert, Supabase-seitig über die integrierte Secrets-Verwaltung (Supabase Vault). Die Abhängigkeiten werden regelmäßig mittels `npm audit` auf bekannte Schwachstellen geprüft; zusätzlich hält der Renovate-Bot die Pakete automatisch aktuell und stellt bei Sicherheitshinweisen zeitnah Update-Pull-Requests bereit.

Die Implementierung erfolgt komponentenbasiert und wird durch automatisierte Tests (Stryker Mutation Testing) sowie Qualitätskontrolle per PageSpeed Insights begleitet. Offene Punkte werden kontinuierlich als Issues dokumentiert und priorisiert abgearbeitet.

1.4 Zielsetzung und Anforderungen

Am Ende soll eine produktiv betriebene, vollständig funktionsfähige Streaming-Landingpage stehen, die öffentlich erreichbar ist und aktiv von der Community genutzt wird. Über den Pilotkanal hinaus muss sie sich – das ist die zentrale Klammer dieses Projekts – allein durch zwei Konfigurationsdateien an einen beliebigen anderen Streamer anpassen lassen.

Funktionale Anforderungen sind die Integration des Twitch-Chats, eine Spenden-Funktion mit vordefinierten Beträgen (Doni-Links), ein Discord-Bot mit Community-Funktionen, ein Twitch-Addon für Stream-Overlays, ein modularer Seitenaufbau nach dem Baukasten-Prinzip, eine Supabase-Datenbankanbindung mit Migrationsverwaltung, eine Authentifizierungs-integration via Single-Sign-On über Twitch sowie eine datenbankgestützte Konfiguration.

Nicht-funktionale Anforderungen umfassen hohe Performance (PageSpeed-Score, nachgewiesen via `pagespeed.web.dev`), Sicherheit (Security-Check, keine sensiblen Daten im Repository), sauberer und kommentierter CSS-Code, Testabdeckung durch Mutation Testing (Stryker) sowie einfache Erweiterbarkeit durch eine modulare Komponentenstruktur.

2 Das Baukasten-Prinzip

Den Kern dieses Projekts bildet nicht die einmalige Umsetzung für einen einzelnen Kanal, sondern der Baukasten-Charakter der Lösung. Sämtliche streamer-spezifischen Inhalte sind konsequent vom Code getrennt und in genau zwei Konfigurationsebenen gebündelt. Wer das System für einen anderen Kanal verwenden möchte, muss ausschließlich diese beiden Ebenen anpassen – an keiner Stelle ist ein Eingriff in die Komponenten-, Hooks- oder Datenbankschicht nötig.

2.1 Konfigurationsebene 1 – `siteConfig.ts`

Die Datei `„src/config/siteConfig.ts“` bündelt sämtliche strukturellen Einstellungen der Seite. Dazu zählen Profilname und Profilbild, der Twitch-Kanalname, die Akzentfarbe (per CSS-Variable in das gesamte Design durchgereicht), die Impressums-Daten, die ICS-URLs des Streamplans inklusive Kategorien, die StreamElements-Donation-URL, sämtliche Link-Karten der Startseite (Haupt-Links, Spiele, Clips, Partner), der OnlyBart-Bereich sowie die Liste interner und externer Weiterleitungen. Auch der Akzentfarbton ist zentral gepflegt; das Frontend liest ihn beim Bootstrap und schreibt ihn als CSS-Variable in das Document-Stylesheet.

Diese strikte Zentralisierung hat zwei Effekte: Erstens lässt sich ein neuer Auftritt in wenigen Minuten konfigurieren – Streamername, Farben, Links und Partner sind jeweils ein Eintrag im Objekt. Zweitens werden hartcodierte Verweise im restlichen Code unmöglich; ein architektonischer Test (vgl. Kapitel 6) prüft genau das.

2.2 Konfigurationsebene 2 – `i18n-Sprachdateien`

Sämtliche Anzeigetexte liegen in den Sprachdateien `„src/i18n/locales/de.json“` und `„src/i18n/locales/en.json“`. Komponenten greifen niemals direkt auf Strings zu, sondern ausschließlich über `i18n`-Schlüssel. So sind Sprache und Tonalität ohne Codeänderung austauschbar – ein neuer Streamer kann seine Tagline, seine Sektionsnamen oder den OnlyBart-Wording ändern, indem er ausschließlich die Sprachdatei anpasst. Ein automatisierter Übersetzungslauf (`„scripts/translate.ts“`) hält die englischsprachige Version mit der deutschen Vorlage synchron.

Diese zweigleisige Struktur – Daten und Verhalten in „siteConfig.ts“, Texte in den i18n-JSONs – ist das definierende Merkmal des Baukastens. Sie ermöglicht es, das Projekt perspektivisch als White-Label-Lösung an weitere Streamer auszuliefern, ohne dass diese selbst Quellcode anfassen müssen.

2.3 Setup-Assistent

Damit der Konfigurationsschritt nicht vollständig händisch erfolgen muss, liegt im Repository der geführte Setup-Assistent „scripts/setup.ts“ bereit, gestartet über „npm run setup“. Er leitet Schritt für Schritt interaktiv durch die Konfiguration – Anzeigenname, Tagline, Akzentfarbe, Impressum, StreamElements-URL und Umgebungsvariablen – und schreibt die Werte am Ende automatisch in „src/config/siteConfig.ts“ sowie in die „.env“-Datei für die lokale Entwicklung. Vorgabewerte lassen sich jeweils per Eingabetaste übernehmen; zusätzliche Werkzeuge werden dabei nicht installiert. Damit auch Nutzer ohne Node-Toolchain den Baukasten konfigurieren können, baut die CI-Pipeline denselben Assistenten zusätzlich als eigenständige „setup.exe“ (über „bun build -compile“) und hängt sie jedem GitHub-Release an; ausgeführt im geklonten Projektverzeichnis schreibt sie dieselben beiden Dateien. Die Anzeigetexte selbst bleiben weiterhin in den i18n-Sprachdateien gepflegt und bilden die zweite Konfigurationsebene.

3 Tech Stack

Für die Umsetzung wurde ein moderner JavaScript- bzw. TypeScript-zentrierter Stack gewählt, der schnelles Prototyping mit produktiver Tauglichkeit verbindet. Im Mittelpunkt stehen React, TypeScript, Vite und Supabase, ergänzt durch Bun für den Twitch-Bot sowie Docker und Nginx als Auslieferungsbasis.

3.1 React & TypeScript

Das Frontend basiert auf React 19 in Verbindung mit TypeScript. React eignet sich besonders für komponentenbasierte Oberflächen und bringt durch Hooks, Context und Suspense alles mit, was für eine reaktive, wartbare Architektur nötig ist. Die strikte Typisierung über TypeScript reduziert Laufzeitfehler, dokumentiert Schnittstellen direkt im Code und macht Refactorings deutlich sicherer – ein wichtiger Faktor angesichts mehrerer feature-lastiger Bereiche wie Voting, OnlyBart und Moderationsdashboard.

3.2 Vite

Als Build-Tool kommt Vite zum Einsatz. Im Vergleich zu klassischen Bundlern wie Webpack liefert Vite im Entwicklungsbetrieb dank ESM-natives-Serving deutlich schnellere Hot-Module-Replacements und ist gleichzeitig für Produktivbuilds mit Rollup vorbereitet. Über die zentrale „vite.config.ts“ werden Code-Splitting und das Lazy-Loading der Routen konfiguriert; Routen wie OnlyBart, ClipVoting oder das Moderationsdashboard werden ausschließlich nachgeladen, wenn sie tatsächlich aufgerufen werden.

3.3 Supabase

Anstelle eines eigenen Backend-Servers wird Supabase als Backend-as-a-Service verwendet. Supabase basiert auf PostgreSQL und stellt Authentifizierung, REST- und Realtime-Endpunkte sowie Storage in einem konsistenten Produkt bereit. Über Row-Level-Security werden Zugriffsrechte direkt in der Datenbank durchgesetzt: Subscriber sehen ausschließlich OnlyBart-Posts, Moderatoren erhalten Schreibrechte auf Voting-Runden, gewöhnliche Nutzer können nur eigene Votes oder Bartclicker-Scores schreiben. Erweiterungen wie „pg_cron“ und „pg_net“ ermöglichen es, geplante Aufgaben (z. B. Voting-Runden-Wechsel) und ausgehende Webhooks direkt aus der Datenbank zu steuern. Ergänzend übernimmt eine Supabase Edge Function das automatisierte Aufräumen abgelaufener Belohnungen.

3.4 Bun (TwitchAddon)

Der Twitch-Channel-Points-Bot ist als eigenständiger Service „TwitchAddon“ in TypeScript implementiert und wird mit Bun ausgeführt. Ursprünglich war eine Java/Spring-Boot-Lösung vorgesehen, da sie für die geplante Twitch-Anbindung über Twitch4J im Java-Ökosystem etabliert ist. In der Praxis erwies sich die Wahl allerdings als unvereinbar mit dem in Kapitel 1.3 formulierten Kostenziel: Eine lauffähige Spring-Boot-Anwendung belegt allein durch die JVM und den Application-Context bereits rund 250 bis 350 MB Arbeitsspeicher im Leerlauf. Auf den verfügbaren kostenlosen Hostings bleibt damit kaum Spielraum für die eigentliche Bot-Logik; die Container wurden in Tests regelmäßig wegen Memory-Überschreitung beendet, und mit der zwischenzeitlichen Abschaffung des Railway-Free-Tiers entfiel ohnehin die naheliegendste Cloud-Option. Im finalen Setup läuft der TwitchAddon daher nicht mehr in der Cloud, sondern als selbst-gepackte Windows-EXE direkt auf dem Streamer-PC; die Erreichbarkeit für die Twitch-Extension wird über einen automatisch beim EXE-Start geöffneten ngrok-Tunnel mit fester Static-Domain hergestellt.

Mit dem Wechsel auf Bun – einer schlanken JavaScript- und TypeScript-Laufzeit – sinkt der RAM-Verbrauch des laufenden Bots auf rund 64 MB. Bun führt TypeScript ohne separaten Build-Schritt aus, startet in unter einer Sekunde und passt damit problemlos in den kostenlosen Tier. Funktional kommuniziert der Bot weiterhin über die Twitch-Helix-API und IRC und schreibt Punktestände sowie Belohnungen direkt in Supabase zurück; lediglich die Laufzeit wurde ausgetauscht. Die Entscheidung dokumentiert exemplarisch, wie das Kostenziel des Projekts auch konkrete Technologie-Entscheidungen geprägt hat.

3.5 Docker & Nginx

Die Auslieferung des Frontends erfolgt wahlweise statisch über GitHub Pages oder als Container hinter einem Nginx-Reverse-Proxy. Das mitgelieferte Dockerfile und die „docker-compose“-Konfiguration kapseln Build und Laufzeit reproduzierbar. Nginx übernimmt Caching der statischen Assets, das Setzen der Sicherheits-Header sowie die SPA-Fallback-Route, sodass tiefe URLs auch beim Direktaufruf korrekt aufgelöst werden. Das TwitchAddon hingegen wird nicht containerisiert, sondern läuft als selbst-gepackte Windows-EXE lokal auf dem Streamer-PC (vgl. Kapitel 3.4).

4 Entwicklung

4.1 Systemarchitektur

Architektonisch besteht das System aus vier deutlich getrennten Bausteinen: dem React-Frontend, dem Supabase-Backend, dem TwitchAddon (Bot + Twitch-Panel-Extension) und dem Discord-Benachrichtigungsdienst, der als serverlose Supabase Edge Function realisiert ist. Die Kommunikation zwischen den Komponenten erfolgt ausschließlich über klar definierte Schnittstellen: REST und Realtime in Richtung Supabase, Twitch-Helix bzw. IRC zwischen TwitchAddon und Twitch, „pg_net“ bzw. Webhooks zwischen Datenbank und dem Discord-Benachrichtigungsdienst. Diese strikte Trennung erlaubt es, einzelne Bausteine unabhängig auszutauschen – eine Voraussetzung, ohne die sich der Baukasten-Gedanke nicht tragen würde.

Die folgende Abbildung zeigt das Komponentendiagramm der Anwendung und stellt die wichtigsten Datenflüsse zwischen den einzelnen Bereichen dar.

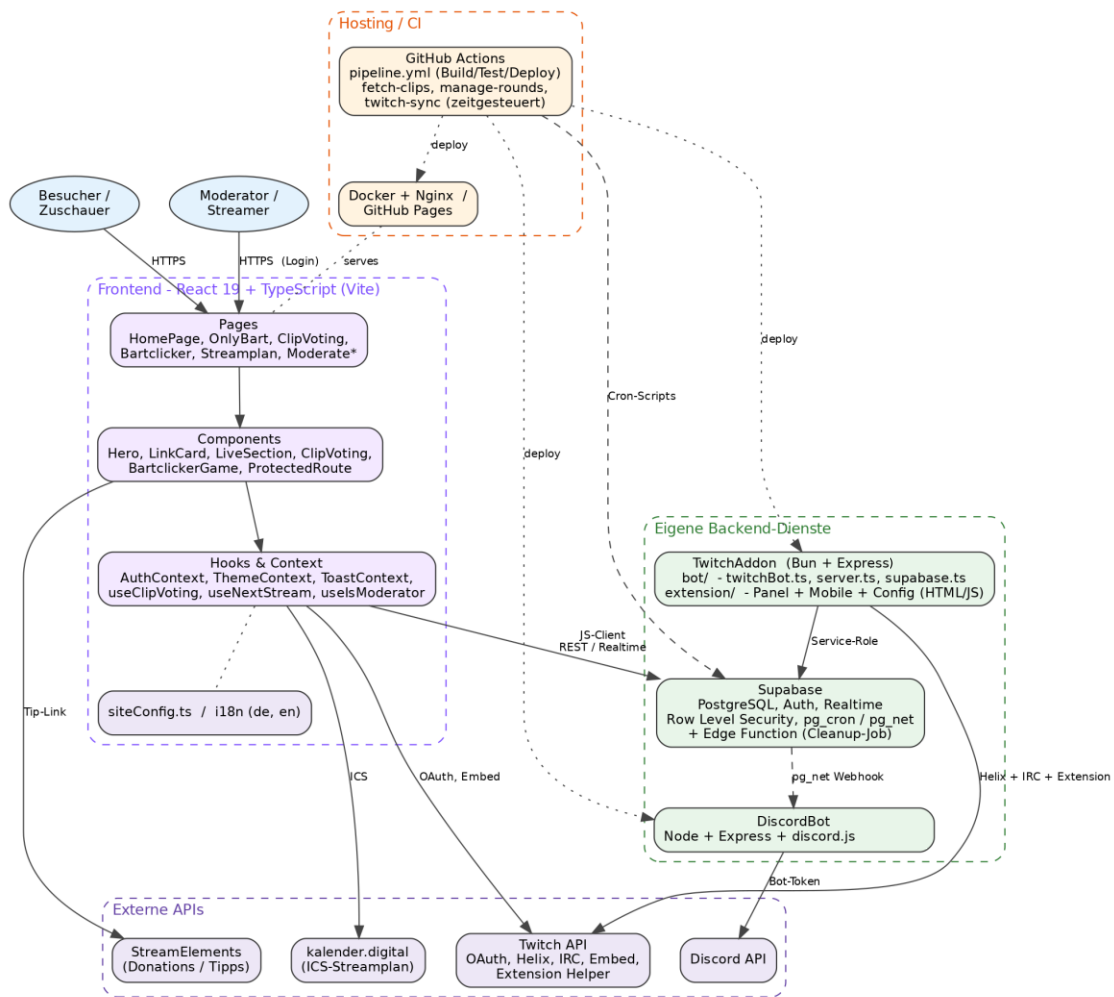


Abb. 1: Komponentendiagramm der Streaming-Landingpage

Abbildung 1 bildet die logische Aufteilung in Frontend, eigene Backend-Dienste, externe APIs sowie Hosting/CI ab. Im oberen Bereich stehen die beiden Akteure (Zuschauer und Moderator/Streamer), die ausschließlich über HTTPS auf das Frontend zugreifen. Das Frontend selbst zerfällt in Pages, Components sowie Hooks und Context-Provider und greift seinerseits auf den konfigurativen Kern (siteConfig.ts und i18n) zu. Die rechts angeordneten Backend-Dienste – Supabase als zentrale Datenbasis, das TwitchAddon und der Discord-Benachrichtigungsdienst – kommunizieren über klar definierte Schnittstellen miteinander. Externe APIs (Twitch, kalender.digital, StreamElements, Discord) sind links zusammengefasst; die Hosting/CI-Ebene steuert das Deployment der einzelnen Bausteine.

4.1.1 Frontend-Aufbau

Das Frontend folgt einer typischen React-Struktur mit drei Ebenen. Die „pages“-Schicht enthält jeweils eine Komponente pro Route (HomePage, OnlyBartPage, ClipVotingPage, BartclickerPage, StreamplanPage, NotFoundPage sowie die Moderate-Seiten). Sämtliche Routen werden in „App.tsx“ über React-Router definiert und per „React.lazy“ asynchron nachgeladen, was die initiale Bundle-Größe gering hält.

Die „components“-Schicht enthält wiederverwendbare UI-Bausteine wie Hero, LinkCard, SectionBox, LiveSection, ClipGrid oder das eigentliche Bartclicker-Spiel. Querschnittsthemen wie Routenschutz (ProtectedRoute, ModeratorRoute, BroadcasterRoute), Mehrsprachigkeit (i18next) und Theming (Light-/Dark-Mode) sind als eigene Komponenten beziehungsweise Context-Provider umgesetzt.

Anwendungslogik und Datenzugriff sind als Custom-Hooks gekapselt: „useClipVoting“ liefert die aktuelle Voting-Runde inklusive Realtime-Updates, „useNextStream“ parst den ICS-Streamplan, „uselsModerator“ und „uselsBanned“ ermitteln die Rolle des angemeldeten Nutzers, „useBartclickerGame“ verwaltet Spiellogik und Punktestand. Diese Hooks isolieren die Abhängigkeit zu Supabase und Twitch und machen die Komponenten leichter testbar.

4.1.2 Backend & Datenmodell

Die Datenhaltung liegt vollständig in Supabase. Innerhalb der Datenbank sind die Inhalte in eigene Schemas aufgeteilt: „clipvoting“ für Clips, Voting-Runden, Stimmen und Gewinner, „onlybart“ für Posts, Likes und Kommentare des Premium-Bereichs sowie „public“ für übergeordnete Tabellen wie Profile, Moderatoren-Liste, Bann-Liste, Bartclicker-Scores, Donation-Trigger und Page-Views.

Sämtliche Tabellen werden über Row-Level-Security abgesichert. Die Policies prüfen anhand der Twitch-User-ID des angemeldeten Nutzers, ob dieser Schreib- oder Leserechte besitzt. Geplante Aufgaben (z. B. das automatische Beenden einer Voting-Runde) sind über „pg_cron“ in der Datenbank hinterlegt; ausgehende Benachrichtigungen werden mittels „pg_net“ als HTTP-POST an die Discord-Edge-Function ausgelöst. Migrationsdateien versionieren das Schema im Verzeichnis „supabase/migrations“.

4.1.3 TwitchAddon (Bot und Panel-Extension)

Das TwitchAddon ist nicht nur ein Chat-Bot, sondern ein zweiteiliger Service. Im Unterordner „bot/“ liegen die modular aufgeteilten Bot-Bestandteile: „twitchBot.ts“ implementiert die IRC- und

Helix-Anbindung, „server.ts“ stellt die HTTP-Endpunkte für die Extension bereit, „supabase.ts“ kapselt den Datenbankzugriff. OAuth-Tokens werden bei Programmstart automatisch erneuert, der Broadcaster wird über die Helix-API ermittelt. Belohnungen, eingelöste Punkte und Donation-Trigger werden über den Service-Role-Schlüssel direkt in Supabase persistiert.

Im Unterordner „extension/“ liegt zusätzlich eine vollwertige Twitch-Panel-Extension. Sie besteht aus drei Ansichten – einem Panel unterhalb des Streams („panel.html“), einer auf Mobilgeräte zugeschnittenen Variante („mobile.html“) sowie einem Konfigurationsdialog für den Streamer („config.html“) – und kommuniziert über den „twitch-ext.min.js“-Helper mit dem laufenden Bot. Zuschauer sehen ihren Punktestand, können das Reward-Shop-Leaderboard einsehen und Belohnungen direkt aus dem Twitch-UI heraus einlösen.

Die Discord-Anbindung erfolgt über die Supabase Edge Function „discord-notify“. Sie wird mit einem Event-Parameter aufgerufen (etwa „start-runde-1“, „ende-runde-1“ oder „start-jahr“) und postet daraufhin eine vorformatierte Nachricht über die Discord-REST-API in den Community-Discord – ein dauerhaft laufender Gateway-Bot ist dafür nicht nötig. Ausgelöst wird die Function wahlweise durch „pg_net“ direkt aus der Datenbank, durch die zeitgesteuerten GitHub-Actions-Jobs oder durch klassische Supabase-Webhooks, sobald sich der Status einer Voting-Runde ändert. Die Authentifizierung erfolgt über ein geheimes Header-Token („x-webhook-secret“), das konstant-zeitig geprüft wird, sodass nur berechtigte Aufrufer Nachrichten anstoßen können. Eine frühere, dauerhaft extern gehostete Bot-Variante wurde damit zugunsten dieses serverlosen, kostenneutralen Ansatzes abgelöst.

4.1.4 Automatisierung über GitHub Actions und Edge Functions

Neben dem klassischen Build-/Deploy-Workflow „pipeline.yml“ existieren in „github/workflows“ drei zeitgesteuerte Jobs, die wiederkehrende Aufgaben automatisieren. „fetch-clips.yml“ holt regelmäßig neue Clips über die Twitch-API und legt sie als Kandidaten für das Voting in Supabase ab; „manage-rounds.yml“ startet und beendet Voting-Runden gemäß dem definierten Monatsrhythmus, einschließlich der Übertragung von Monats- in Jahresgewinner; „twitch-sync.yml“ aktualisiert Profilinformationen wie Anzeigenamen oder Avatare. Die zugehörigen TypeScript-Skripte liegen in „github/scripts“ und werden in den Workflows ausgeführt.

Ergänzend übernimmt eine Reihe von Supabase Edge Functions serverlose Aufgaben, die weder im Browser noch in einem dauerhaft laufenden Server gut aufgehoben sind: „discord-notify“ verschickt die Voting-Benachrichtigungen, „calendar“ liefert den Streamplan stets aktuell als ICS-Proxy zu kalender.digital (mit Host-Allowlist gegen SSRF), „twitch-user“ löst Twitch-Logins serverseitig über die Helix-API in Nutzer-IDs auf, „twitch-game“ und „check-stores“

ermitteln das aktuell gespielte Spiel samt passender Store-Links, und „delete_expired_redeemed_global“ entfernt periodisch abgelaufene globale Belohnungen. So verteilen sich Automatisierungsaufgaben auf drei sinnvoll getrennte Ebenen: GitHub Actions für externe Integrationen, „pg_cron“/„pg_net“ in Supabase für DB-nahe Trigger und Edge Functions für serverlose Logik. Sämtliche Edge Functions sind dabei optional ausgelegt – fehlt eine, fällt das Frontend auf einen statischen Stand oder einen Drittanbieter zurück, sodass die Seite stets funktionsfähig bleibt.

4.2 Design

Das visuelle Design ist bewusst klar und reduziert gehalten. Die zentrale Akzentfarbe (per CSS-Variable „--accent“ aus „siteConfig.ts“) zieht sich durch Buttons, Karten und Hover-Effekte. Sämtliche Bereiche sind responsiv gestaltet; auf Mobilgeräten schaltet eine Toggle-Bar zwischen den Sektionen „Live“, „Links“, „Spiele“ und „Partner“ um, sodass die Inhalte auf kleinen Displays nicht überladen wirken. Ein Light-/Dark-Mode wird über „ThemeContext“ gesetzt und im LocalStorage persistiert. Animationen (z. B. Live-Indikator-Pulsation, Mascot der 404-Seite, Bartclicker-Übergänge) werden über die Bibliothek „motion“ deklarativ in den Komponenten beschrieben.

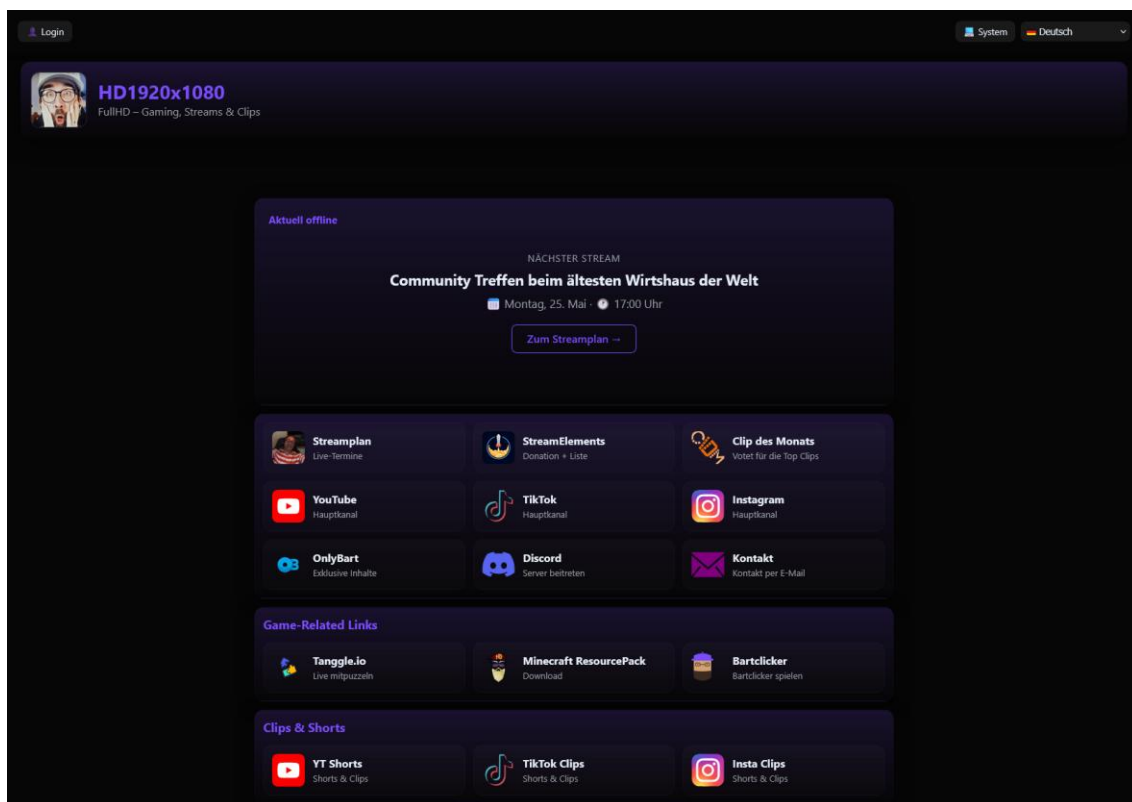


Abb. 2: Startseite des Pilotkanals hd1920x1080

Abbildung 2 zeigt die produktiv betriebene Startseite des Pilotkanals und macht die zuvor beschriebenen Designprinzipien sichtbar. Oben befindet sich die globale Settings-Bar mit Login, Theme-Umschalter und Sprachwahl. Direkt darunter blendet das Hero-Element Profilbild, Anzeigenname und Tagline ein – sämtliche Inhalte werden dabei aus „siteConfig.ts“ und den i18n-Sprachdateien gespeist. Die Live-Section meldet den aktuellen Stream-Status und zeigt im Offline-Fall den nächsten Termin samt „Zum Streamplan“-Button an. Die folgenden Sektionen – Haupt-Links, Game-Related Links, Clips & Shorts – werden aus den entsprechenden Listen in der Konfiguration generiert; jede LinkCard bezieht Titel, Beschreibung und Icon ausschließlich aus diesen beiden Konfigurationsebenen. Damit illustriert der Screenshot zugleich, wie konsequent das in Kapitel 2 beschriebene Baukasten-Prinzip in der finalen Oberfläche umgesetzt ist.

4.3 Hosting und CI/CD-Pipeline

Der Build- und Deploy-Prozess wird vollständig über GitHub Actions abgewickelt. Die zentrale Workflow-Datei „pipeline.yml“ definiert eine fünfstufige Pipeline, deren einzelne Stages voneinander abhängen und teilweise parallelisiert ausgeführt werden.

In Stage 1 (compute-version) berechnet ein eigenes Skript die nächste Versionsnummer anhand der Conventional-Commit-Konvention: Commits mit „feat“ erhöhen die Minor-, alle übrigen die Patch-Stelle, mit „BREAKING CHANGE“ markierte Commits lösen einen Major-Bump aus. Stage 2 (validate) installiert die Abhängigkeiten und führt für beide Teilprojekte (Landingpage und TwitchAddon) Linter, TypeScript-Compiler und Vitest-Tests aus; im selben Schritt laufen die in Kapitel 5.4 beschriebenen Sicherheitsprüfungen (npm audit und Retire.js).

Stage 3 (release-twitchaddon) baut ausschließlich auf den Branches main, master und dev eine standalone Windows-EXE des TwitchAddons sowie – aus demselben „bun build --compile“ – die eigenständige „setup.exe“ des Konfigurations-Assistenten. Beide werden zusammen mit der mitgelieferten ngrok-EXE und der gepackten Twitch-Extension als GitHub-Release veröffentlicht. Stage 4 (docker) erzeugt das Container-Image für das Frontend und pusht es in die GitHub Container Registry. Stage 5 (deploy-pages) veröffentlicht den fertigen Frontend-Build schließlich auf GitHub Pages.

Das Frontend wird wahlweise statisch auf GitHub Pages oder als Docker-Image hinter einem Nginx-Reverse-Proxy bereitgestellt. Insgesamt fallen damit – wie in Kapitel 1 angekündigt – außer der Domain keine laufenden Kosten an.

5 Tests

5.1 Unit-Tests

Zur Sicherung der Codequalität sind Unit-Tests mit Vitest und der React Testing Library implementiert. Getestet werden insbesondere die wiederverwendbaren UI-Komponenten (Hero, Footer, LinkCard, SectionBox, CookieBanner) sowie zentrale Hilfsfunktionen aus „src/lib/utils.ts“. Die Tests prüfen sowohl das gerenderte Markup als auch das Verhalten auf Nutzerinteraktion, etwa das Akzeptieren des Cookie-Banners oder das Auslösen eines Downloads über die LinkCard.

Zusätzlich existiert ein architektonischer Test („src/test/arch.test.ts“), der sicherstellt, dass keine zyklischen Importe entstehen und dass keine Komponente hartcodierte Streamer-Daten enthält – ein direkter Schutz für den Baukasten-Gedanken. Ergänzend wird über Stryker ein Mutation-Testing-Lauf ausgeführt, der die Aussagekraft der bestehenden Tests prüft, indem er Codestellen gezielt verändert und kontrolliert, ob die Tests die Änderung bemerken.

5.2 User-Tests

Parallel zu den automatisierten Tests wurden mehrere informelle Usability-Tests mit Mitgliedern der Twitch-Community durchgeführt. Beobachtet wurde, wie schnell Nutzerinnen und Nutzer den nächsten Streamtermin finden, wie intuitiv das Clip-Voting funktioniert, ob die Anmeldung über Twitch klar erkennbar ist und wie sich die Seite auf verschiedenen Mobilgeräten verhält.

Aus den Rückmeldungen ergaben sich mehrere Anpassungen: Die Live-Section wurde sichtbarer an den Anfang der Seite verschoben, der Mobile-Toggle bekam stärker abgegrenzte Tabs, die Ladezustände wurden um Skeleton-Loader ergänzt und die Texte des Cookie-Banners wurden gekürzt. Auch technische Schwächen, etwa eine fehlerhafte Behandlung abgelaufener Twitch-Tokens, wurden im Zuge dieser Tests aufgedeckt und behoben.

5.3 Performance-Prüfung mit PageSpeed Insights

Die in Kapitel 1 definierte Performance-Anforderung wird kontinuierlich über Google PageSpeed Insights abgesichert. PageSpeed Insights bewertet eine Seite anhand von vier Kategorien – Performance, Accessibility, Best Practices und SEO – jeweils auf einer Skala von 0 bis 100, getrennt für Mobile- und Desktop-Aufrufe. Grundlage sind synthetische Lighthouse-Messungen mit Metriken wie First Contentful Paint, Largest Contentful Paint, Total Blocking Time und Cumulative Layout Shift.

Bei der am 11.06.2026 durchgeführten Analyse für `hd1920x1080.de` wurden folgende Werte ermittelt:

	Mobile	Computer
Performance:	98/100	100/100
Accessibility:	100/100	100/100
Best Practices:	100/100	100/100
SEO:	100/100	100/100

Die Werte werden gehalten durch (1) das Lazy-Loading aller Routen via „React.lazy“, wodurch das initiale JavaScript-Bundle klein bleibt, (2) WebP/SVG-Vorabkomprimierung der Bilder im `public`-Ordner, (3) Cache-Header und gzip-Kompression durch Nginx sowie (4) den weitgehenden Verzicht auf Drittanbieter-Skripte. Der vollständige Report ist öffentlich unter https://pagespeed.web.dev/analysis/https-hd1920x1080-de/jp4dn0xxcf?form_factor=mobile erreichbar.

5.4 Sicherheitsprüfung

Die in Kapitel 1.3 angekündigten Sicherheitsmaßnahmen werden über drei ineinandergreifende, weitgehend automatisierte Prüfverfahren umgesetzt. Sie laufen teils lokal, teils in der CI-Pipeline und teils kontinuierlich im Hintergrund.

5.4.1 npm audit

„npm audit“ ist das in npm integrierte Werkzeug zur Schwachstellenprüfung. Es vergleicht alle direkten und transitiven Abhängigkeiten der „package-lock.json“ gegen die GitHub Advisory Database und meldet bekannte CVE-Einträge nach Schweregrad. In der Pipeline („pipeline.yml“) wird der Befehl mit der Option „--audit-level=high“ ausgeführt; ein Treffer ab dieser Schwelle bricht den Build ab und verhindert ein Deployment mit bekannt verwundbaren Paketen.

5.4.2 Retire.js

Während sich „npm audit“ am Dependency-Tree orientiert, untersucht „Retire.js“ den tatsächlich gebündelten JavaScript-Code auf veraltete Bibliotheksversionen. Das ist insbesondere bei eingebetteten Skripten relevant, die nicht über npm verwaltet werden. In der Pipeline läuft

Retire.js auf den gesamten Projektordner, schreibt einen JSON-Report und lädt diesen als Build-Artefakt hoch. Auch hier bricht der Build bei Befunden ab Schweregrad „high“ ab.

5.4.3 Renovate

Die kontinuierliche Aktualisierung der Abhängigkeiten übernimmt der Renovate-Bot. Er deckt sowohl reguläre Versionsupdates als auch sicherheitsrelevante Aktualisierungen ab und ersetzt damit eine separate, rein sicherheitsbezogene Lösung vollständig. Die zugehörige Konfiguration liegt in „renovate.json“ und basiert auf dem empfohlenen Profil „config:recommended“. Patch- und Minor-Updates werden automatisch zusammengeführt („platformAutomerge“), sobald CI grün ist; Major-Updates erfordern eine explizite manuelle Freigabe, weil sie Breaking-Changes enthalten können. Über „prHourlyLimit: 0“ und „prConcurrentLimit: 0“ ist die Anzahl der parallelen Update-PRs nicht künstlich beschränkt, was in Kombination mit dem automatisierten Merge eine sehr kurze Update-Latenz ermöglicht. Sicherheitsrelevante Aktualisierungen werden so meist innerhalb eines Tages produktiv.

6 Setup

Das Setup spiegelt den Baukasten-Gedanken wider und wurde von Beginn an so gestaltet, dass das gesamte Projekt mit wenigen Schritten reproduzierbar in Betrieb genommen werden kann. Im einfachsten Fall genügen ein Fork des Repositories sowie ein Lauf von „npm install“ und „npm run dev“, um die Landingpage lokal zu starten.

Für den unmittelbaren Einsatz steht der bereits in Kapitel 2.3 vorgestellte Setup-Assistent bereit – wahlweise per „npm run setup“ aus dem Quellcode oder als vorgefertigte „setup.exe“ aus dem GitHub-Release, falls keine Node-Toolchain vorhanden ist. Er führt durch die strukturelle Konfiguration und schreibt das Ergebnis in „src/config/siteConfig.ts“ und die „.env“-Datei, sodass die lokale Instanz unmittelbar startklar ist.

Im produktiven Betrieb wird das Frontend per GitHub Actions gebaut und entweder direkt auf GitHub Pages oder über Docker und Nginx hinter einer eigenen Domain ausgeliefert. Für den Anschluss an Supabase wird ein Projekt angelegt, die mitgelieferten SQL-Migrationen werden ausgeführt und die zugehörigen Schlüssel in den GitHub-Secrets hinterlegt. Die Twitch-Panel-Extension aus „TwitchAddon/extension“ wird im Twitch-Developer-Dashboard registriert; als Fetch-URL trägt man dort die feste ngrok-Static-Domain ein, die der TwitchAddon beim Start automatisch öffnet.

7 Projektabschluss

7.1 Fazit

Die Entwicklung der Streaming-Landingpage hat gezeigt, dass sich eine umfangreiche, community-orientierte Plattform mit einem schlanken Stack aus React, TypeScript, Vite und Supabase mit verhältnismäßig geringem Aufwand professionell umsetzen lässt. Sämtliche Kernfunktionen – Linkpage, Streamplan, Live-Section, Clip-Voting, OnlyBart, Bartclicker, Twitch-Panel-Extension sowie das Moderatoren-Dashboard – wurden vollständig implementiert und produktiv in Betrieb genommen. Wichtiger als diese Einzelfunktionen ist jedoch der Gesamtaufbau: Das Projekt ist nicht eine Webseite, sondern ein Baukasten, dessen Wirkung sich erst in der Wiederverwendung entfaltet.

Besonders die Entscheidung, das Backend nicht selbst zu betreiben, sondern auf Supabase mit Row-Level-Security zu setzen, hat sich als tragfähig erwiesen. Sie ermöglicht es, sensible Bereiche wie OnlyBart oder das Moderationsdashboard sauber abzusichern, ohne eine eigene Auth-Infrastruktur pflegen zu müssen. Auch das Kostenziel aus Kapitel 1 wurde erreicht: Bis auf die Domain entstehen keine laufenden Kosten.

7.2 Herausforderungen

Eine zentrale Herausforderung war die Authentifizierung mit Twitch. Die OAuth-Flows müssen sowohl im Frontend (für Nutzer-Login) als auch im TwitchAddon (für den Bot-Token) stabil mit ablaufenden Tokens umgehen. Hier waren mehrere Iterationen nötig, bis automatische Refresh-Mechanismen und ein klar definierter Fehlerpfad bestanden.

Eine zweite Hürde stellte die Modellierung der Voting-Logik dar. Monatliche Runden, ein abschließendes Jahresvoting, Stimmkontingente pro Nutzer sowie die Übertragung von Monats- in Jahresgewinner mussten konsistent in der Datenbank abgebildet werden. Die Kombination aus Datenbank-Triggern, „pg_cron“-Jobs, GitHub-Actions-Cron-Skripten und Webhooks zur Discord-Edge-Function machte es nötig, viele Sonderfälle gezielt zu testen.

Schließlich war die konsequente Konfigurierbarkeit – also der eigentliche Baukasten-Anspruch – ein wiederkehrender Schmerzpunkt. Damit das Projekt als wiederverwendbare Vorlage funktioniert, mussten auch hartcodierte Werte wie URLs, Channel-Namen oder Discord-Webhook-Endpunkte konsequent in die zentrale Konfiguration ausgelagert werden. Erst nach mehreren Refactorings und der Einführung des architektonischen Tests konnte sichergestellt werden, dass ein Streamer-Wechsel ausschließlich über „siteConfig.ts“ und die Sprachdateien möglich ist.

7.3 Zukunftspotential

Das Projekt bietet eine Vielzahl an Erweiterungsoptionen. Naheliegend ist der Ausbau des OnlyBart-Bereichs zu einer vollwertigen Premium-Plattform mit Tier-Abstufungen, Push-Benachrichtigungen oder einer eigenen Mobile-Wrapper-App. Auch das Bartclicker-Spiel lässt sich um Mehrspielermodus, saisonale Events und Twitch-Drops erweitern.

Mittelfristig ist der konsequente nächste Schritt ein Marketplace-Ansatz, bei dem der Baukasten als White-Label-Lösung an weitere Streamer ausgeliefert wird. Da alle streamer-spezifischen Daten bereits in zwei Konfigurationsebenen gebündelt sind, fehlt für diesen Schritt im Wesentlichen ein geführter Onboarding-Wizard, der ein neues Supabase-Projekt automatisch provisioniert und die beiden Konfigurationsdateien für den neuen Streamer befüllt. Die Architektur ist auf genau dieses Szenario ausgelegt.

8 Eigenständigkeitserklärung

Hiermit bestätige ich, dass ich die vorliegende Arbeit selbstständig verfasst und keine anderen Publikationen, Vorlagen und Hilfsmittel (z. B. künstliche Intelligenz) als die angegebenen benutzt habe. Alle Teile meiner Arbeit, die wortwörtlich oder dem Sinn nach anderen Werken entnommen sind, wurden unter Angabe der Quelle kenntlich gemacht. Gleiches gilt für von mir verwendete Internetquellen. Ich versichere, dass ich diese Arbeit oder nicht zitierte Teile daraus vorher nicht in einem anderen Prüfungsverfahren eingereicht habe. Mir ist bekannt, dass meine Arbeit zum Zwecke eines Plagiatsabgleichs mittels einer Plagiatserkennungssoftware auf eine ungekennzeichnete Übernahme von fremdem geistigem Eigentum, sowie auf die Nutzung von künstlicher Intelligenz zur Texterstellung, überprüft werden kann.

I hereby confirm that I have independently written this work and have not used any publications, templates, or aids (e.g. artificial intelligence) other than those I have indicated. All parts of my work which have been taken literally or correspondingly from other publications have been duly acknowledged. This also applies to Internet sources. I confirm that I have not previously submitted this work or any unquoted parts thereof in any other examination procedure. I am aware that my work may be checked for plagiarism by means of plagiarism recognition software, as well as for the use of artificial intelligence for text creation, in order to verify the integrity of its written content.

Bergisch Gladbach, 19.5.2026

A handwritten signature in black ink, appearing to read 'R. Wagner'. The signature is stylized with a large, looped 'R' and a cursive 'Wagner'.